Assignment 1

TASK 1 – Understand the Problem and Identify the Objects

1. Object-Oriented Analysis

The Namma Metro Smart-Card Fare System calculates fares based on the zone travelled by the passenger. It contains three zones: Inner, Middle and Outer. The fares are Rs. 10, Rs. 20 and Rs. 30 respectively. During peak hours (8–10 AM and 5–8 PM), a 20% surcharge is applied. Students and senior citizens receive concessions. A passenger can travel only if at least Rs. 50 remains in the smart card after fare deduction.

2. Objects, Data and Behaviour

Object	**Data (Attributes)**	**Behaviour (Methods)**
SmartCard	Card ID, balance	Check balance, deduct fare, recharge
Passenger	Name, passenger type	Select journey, receive concession
Journey	Source, destination, zone, peak status	Calculate fare, validate journey
Zone	Zone name, base fare	Identify zone fare
FareTable	Zone fares, surcharge percentage	Return fare, apply surcharge

3. Procedural and Object-Oriented Programming

Procedural Programming:

In procedural programming, separate variables and functions are used to calculate fares and maintain card balances.

Example:

- Variables: balance, fare, zone, peakHour
- Functions: calculateFare(), checkBalance(), deductFare()

Object-Oriented Programming:

In object-oriented programming, data and functions are combined into classes.

Example:

- SmartCard class stores balance and deducts fare.
- Journey class stores journey details.
- FareTable class calculates fare.

OOP makes the program easier to maintain, modify and reuse.

4. Basic Principles of OOP

Encapsulation: Combines data and methods inside a class. The SmartCard balance can be private and accessed through methods.

Abstraction: Hides internal implementation details. Passengers only need to tap the card without knowing the internal fare calculation.

Inheritance: StudentPassenger and SeniorCitizenPassenger can inherit from the Passenger class.

Polymorphism: The same concession function can provide different concessions for students and senior citizens.

5. Rules of a Correct Journey

1. The smart card must be valid.
2. The zone must be valid.
3. Fare must be calculated according to the zone.
4. A 20% surcharge must be applied during peak hours.
5. Student and senior citizen concessions must be applied correctly.
6. Fare must be deducted only when the journey is allowed.

7. At least Rs. 50 must remain after fare deduction.
8. The balance must never become negative.
9. Fare calculations must be accurate.
10. Peak hours must be checked correctly.

6. Alternative Concept and Complexity Analysis

If the system is implemented using procedural programming, separate variables and functions can be used instead of SmartCard, Journey and FareTable classes.

For example:

- balance stores the card balance.
- calculateFare() calculates the fare.
- canTravel() checks whether the journey is allowed.
- deductFare() deducts the fare.

Operation	**Procedural**	**OOP**
Fare calculation	O(1)	O(1)
Balance checking	O(1)	O(1)
Memory per card	O(1)	O(1)
Time per journey	O(1)	O(1)

Both approaches have the same time and space complexity. However, OOP is easier to maintain because data and functions are organized into classes. If the minimum balance changes, the OOP design can update one constant or method instead of modifying multiple functions.

7. Conclusion

The object-oriented approach is suitable for the Metro Smart-Card Fare System because it provides encapsulation, abstraction, inheritance and polymorphism. It improves data security,

code reusability and maintainability. Therefore, OOP is recommended for implementing a reliable metro fare system.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

1. Introduction

In the Metro Smart-Card Fare System, constants are used to define fares, surcharge percentages and minimum balance limits. Using constants helps avoid repeated values and makes the program easier to maintain.

2. Five Types of Tokens in C++

Tokens are the smallest individual elements of a C++ program.

Token Type	**Example**	**Explanation**
Keywords	int, class, switch, return	Reserved words in C++
Identifiers	balance, fare, SmartCard	Names given to variables, classes and functions
Constants	1000, 2000, 3000	Fixed values that do not change
Operators	+, -, %, ==	Perform operations
Punctuators	;, {}, (), []	Define program structure

3. #define, const and constexpr

#define: It is a preprocessor directive used to define symbolic constants. It does not provide type checking.

Example: #define BASE_FARE 1000

const: It declares a variable whose value cannot be modified after initialization.

Example: const int MIN_BALANCE = 5000;

constexpr: It defines a constant expression that can be evaluated at compile time.

Example: constexpr int PEAK_SURCHARGE_PERCENT = 20;

Why constexpr is preferred:

- Provides type safety.
- Supports compile-time evaluation.
- Helps avoid repeated fare values.
- Makes fare changes easier.
- Suitable for defining fixed fare tables.

4. enum and enum class

enum: Defines named constants representing different zones.

Example: enum Zone { Inner, Middle, Outer };

enum class: Defines strongly typed enumerations.

Example: enum class Zone { Inner, Middle, Outer };

Why enum class is safer:

- Prevents accidental conversion to integers.
- Avoids naming conflicts.
- Requires explicit use of the enum type.
- Improves readability and type safety.

5. Naming Rules and Conventions

Identifiers must follow these rules:

1. An identifier must begin with a letter or underscore.
2. It cannot begin with a digit.

3. Keywords cannot be used as identifiers.
4. Spaces are not allowed.
5. Identifiers are case-sensitive.

Examples:

- balance – variable name
- calculateFare() – function name
- SmartCard – class name
- MIN_BALANCE_PAISE – constant name

6. Alternative Concept and Complexity Analysis

If fare values are defined using #define instead of constexpr and enum class, the program can still calculate fares.

Feature	**#define**	**constexpr**
Type checking	No	Yes
Scope	No normal variable scope	Normal C++ scope
Memory	Usually no storage	Usually no storage
Execution time	No extra runtime cost	No extra runtime cost
Safety	Less safe	More safe

Both approaches have the same running time for fare calculation: **O(1)**.

7. Conclusion

Using constexpr and enum class is recommended for the Metro Smart-Card Fare System because they provide type safety, readability and easier maintenance. Constants ensure that fare values and surcharge percentages are defined in one place only.

TASK 4 – Control Implicit Conversionsyt4

Code execution required

This task includes a Predict the Output program. First, predict the output yourself, then execute it and explain the actual output. The program below is taken from your assignment PDF.

1. Introduction

Implicit conversion occurs when the C++ compiler automatically converts one data type into another without an explicit instruction from the programmer.

2. Implicit Conversions in C++

a) Integer Division and Integer-to-Double Conversion

When two integers are divided, the fractional part is removed.

Example:

7 / 2 = 3

When one operand is converted into double:

7.0 / 2 = 3.5

b) Signed and Unsigned Comparison

When a negative signed integer is compared with an unsigned integer, the signed value may be converted into unsigned.

This can produce unexpected results.

c) Narrowing Conversion

Narrowing conversion occurs when a value is converted into a type that cannot represent it accurately.

Example:

int x{3.7};

This produces a compilation error because brace initialization prevents narrowing conversions.

d) static_cast

static_cast is used to perform explicit type conversion.

Example:

double result = static_cast<double>(7) / 2;

Output:

3.5

3. Predict the Output

Before executing the program, write your prediction in your assignment.

4. C++ Program

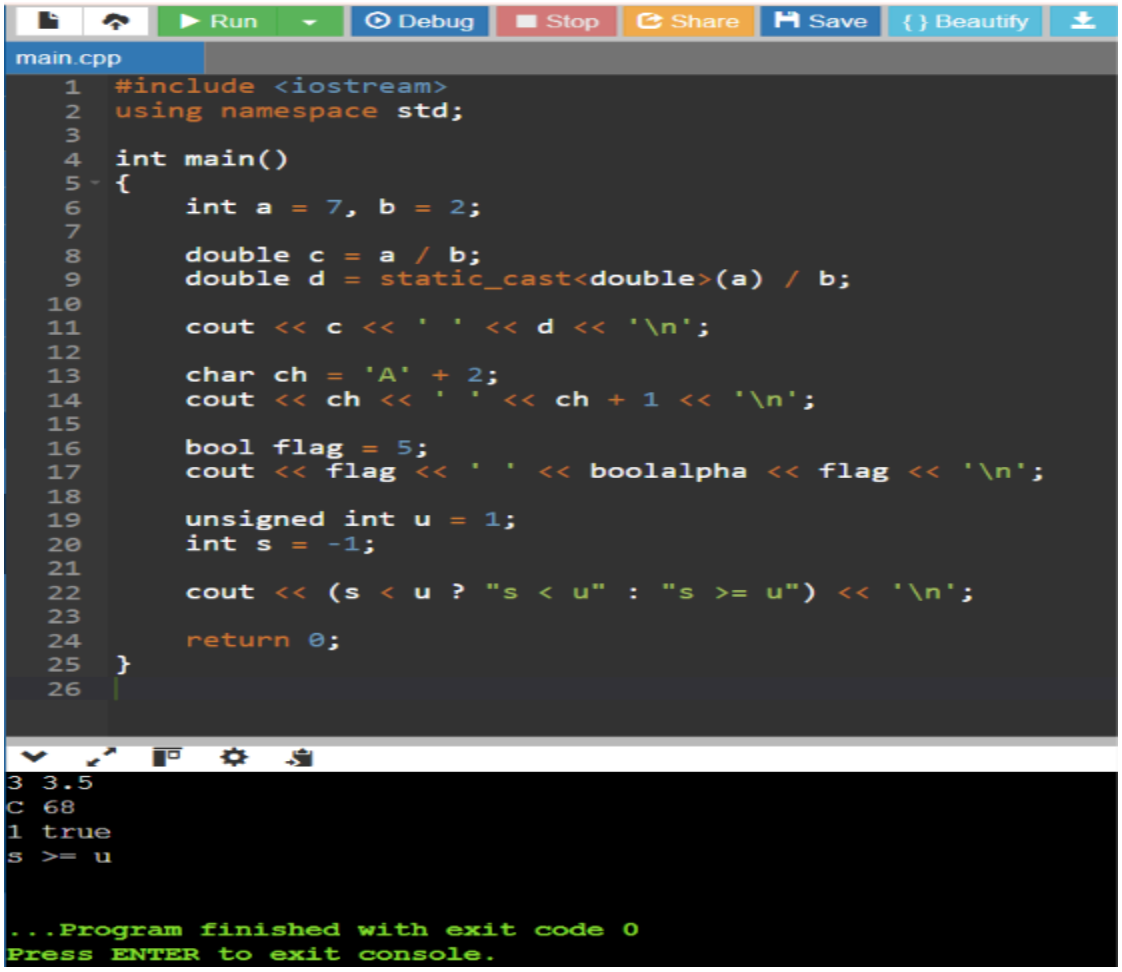
```
None

#include <iostream>
using namespace std;
int main()
{
   int a = 7, b = 2;

   double c = a / b;
   double d = static_cast<double>(a) / b;
   cout << c << ' ' << d << '\n';
```

```
    char ch = 'A' + 2;
    cout << ch << ' ' << ch + 1 << '\n';
    bool flag = 5;
    cout << flag << ' ' << boolalpha << flag << '\n';
    unsigned int u = 1;
    int s = -1;
    cout << (s < u ? "s < u" : "s >= u") << '\n';
    return 0;
}
```

Output



6. Explanation of Each Output

Output	Explanation
3 3.5	a / b performs integer division, giving 3. static_cast<double>(a) / b gives 3.5.
C 68	'A' + 2 gives ASCII value 67, which represents C. ch + 1 promotes char to int, giving 68.
1 true	Integer 5 converts to bool true. By default, true prints as 1. boolalpha prints it as true.
s >= u	The signed −1 converts to unsigned, becoming a large unsigned value. Therefore, the comparison is false for s < u.

7. Alternative Concept and Complexity Analysis

If all conversions are written explicitly using static_cast and brace initialization, the program becomes safer and easier to understand.

Example:

```
None
double d = static_cast<double>(a) / b;
int x{3};
```

- Explicit conversion improves readability.
- Brace initialization prevents narrowing errors.

- Some mistakes are detected during compilation.
- Running time remains O(1).
- Memory usage remains unchanged.

Feature	**Implicit Conversion**	**Explicit Conversion**
Conversion	Automatic	Programmer specifies
Readability	Sometimes confusing	More clear
Safety	May cause unexpected results	Better control
Time complexity	O(1)	O(1)
Memory	Same	Same

8. Conclusion

Implicit conversions can cause unexpected results in C++, especially in integer division, signed and unsigned comparisons, and narrowing conversions. Using static_cast and brace initialization improves program safety and correctness.

TASK 5 – Write Correct Expressions with the Right Precedence

Code execution required

According to your assignment, Task 5 includes a Predict the Output program. You must predict the output, execute the program, and explain operator precedence and associativity.

1. Introduction

Operator precedence determines the order in which operators are evaluated in C++. Associativity determines the direction of evaluation when operators have the same precedence.

2. Expressions Used in Metro Fare System

Peak Hour Check

```
None
(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)
```

This checks whether the hour is between 8–10 AM or 5–8 PM.

Surcharge Calculation

```
None
fare += fare * 20 / 100;
```

This adds a 20% surcharge to the fare during peak hours.

Concession Calculation

```
None
fare = fare - (fare * concession / 100);
```

This subtracts the concession amount from the fare.

3. Operator Precedence

- *, /, % have higher precedence than + and -.
- Relational operators such as > have higher precedence than &&.

- && has higher precedence than ||.
- The conditional operator ?: evaluates the condition first.
- Compound assignment operators such as += update the variable.
- x++ uses the old value and then increments.
- ++x increments first and then uses the new value.

4. Difference Between x++ and ++x

- x++ is post-increment.
- ++x is pre-increment.

Example:

```
None
int x = 5;
int a = x++;
```

Here, a = 5 and x = 6.

```
None
int x = 5;
int a = ++x;
```

Here, a = 6 and x = 6.

i = i++ should be avoided because it can cause confusing and error-prone behaviour.

5. Predict the Output

Before execution, write your predicted output in the assignment.

6. C++ Program

Using namespace std:

None

```
#include <iostream>
using namespace std;

int main()
{
   int x = 5, y = 3;

   cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

   int z = x++ + ++y;
   cout << z << ' ' << x << ' ' << y << '\n';

   cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';

   cout << 10 / 3 << ' ' << 10 % 3 << ' '
        << -10 / 3 << ' ' << -10 % 3 << '\n';

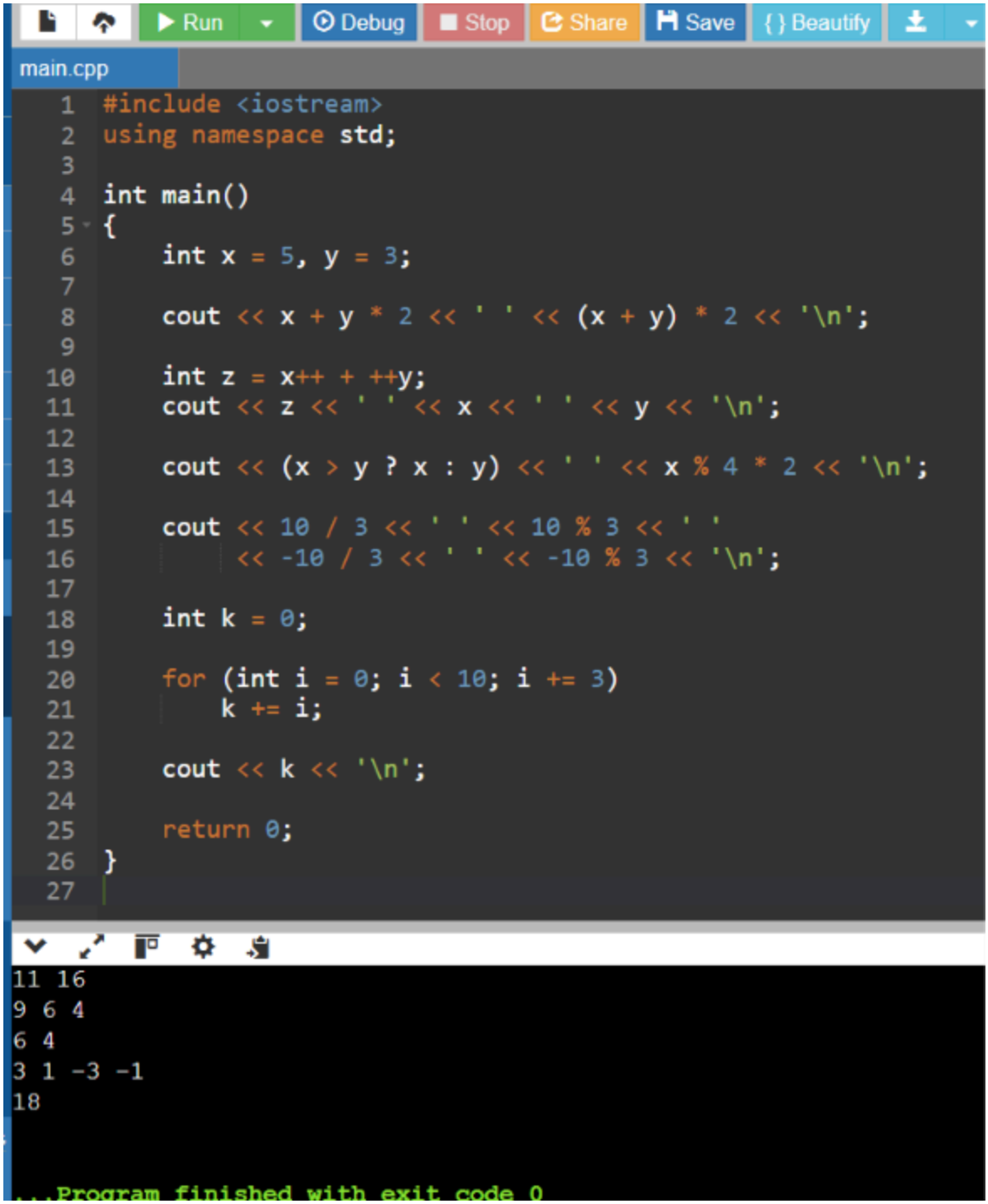
   int k = 0;

   for (int i = 0; i < 10; i += 3)
      k += i;

   cout << k << '\n';

   return 0;
}
```

Output



8. Explanation of Output

Output	Explanation

11 16	Multiplication has higher precedence than addition. Parentheses change the order.
9 6 4	Post-increment uses old x value; pre-increment increases y first.
6 4	Conditional operator returns x because x > y. Modulus is evaluated before multiplication.
3 1 -3 -1	Integer division removes decimal part. Negative division truncates toward zero.
18	Loop adds 0 + 3 + 6 + 9 = 18.

9. Alternative Concept and Complexity Analysis

If peak-hour checking is performed using a lookup table bool peak[24] instead of && and || expressions:

- The hour is used as an array index.
- Peak status is directly obtained from the table.
- Both methods have O(1) time complexity.
- The lookup table requires 24 bytes if bool occupies 1 byte.
- The expression method does not require an additional array.

10. Conclusion

Operator precedence and associativity are important for correct fare calculation. Parentheses improve readability, and understanding pre-increment, post-increment, division and modulus helps avoid errors.

TASK 6 – Design the Control Structures

1. Introduction

Control structures are used to make decisions and repeat operations in the Metro Smart-Card Fare System. They help calculate fares, validate journeys and process multiple passengers.

2. Control Structures

a) if-else

Used to check conditions and make decisions.

Example:

- If the passenger has sufficient balance, allow travel.
- Otherwise, deny travel.

b) switch

Used to select a base fare according to the zone.

- Inner Zone – Rs. 10
- Middle Zone – Rs. 20
- Outer Zone – Rs. 30

c) Conditional Operator

The conditional operator ?: is a short form of if-else.

Example:

```
None
fare = peak ? fare + fare * 20 / 100 : fare;
```

d) Loops

- for loop: Suitable when the number of journeys is known.
- Range-based for loop: Suitable for processing an array of journeys.
- while loop: Suitable when journeys continue until a condition becomes false.

A range-based for loop is preferred for processing a collection of journeys because it is simple and readable.

e) break and continue

- break terminates a loop or switch statement.
- continue skips the current iteration and moves to the next iteration.

If an invalid journey is found, continue can skip that journey.

f) No Matching switch Case

If no case matches, the default case is executed. If no default case exists, the switch statement does nothing.

3. Algorithmic Pseudocode

```
None

Algorithm: Calculate Metro Fare


1. Start
2. Read zone, peak status and card balance
3. Select base fare using zone
    Inner  -> 1000 paise
    Middle -> 2000 paise
    Outer  -> 3000 paise
4. If peak hour is true
    surcharge = fare * 20 / 100
    fare = fare + surcharge
```

```
5. Check balance - fare >= 5000
6. If condition is true
      Deduct fare from balance
      Display fare and remaining balance
   Else
      Display "Denied (low balance)"
7. Stop
```

4. Complete the Code – TODO 1 and TODO 2

Use this code in your C++ compiler.

```
None

#include <iomanip>
#include <iostream>
using namespace std;

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base = 0;

    switch (z)
    {
```

```
        case Zone::Inner:
            base = BASE_PAISE[0];
            break;
        case Zone::Middle:
            base = BASE_PAISE[1];
            break;
        case Zone::Outer:
            base = BASE_PAISE[2];
            break;

        default:
            return 0;
    }
    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;

    return base;
}
bool canTravel(long long balance, long long f)
{
    return (balance - f >= MIN_BALANCE_PAISE);
}
int main()
{
    long long balance = 10000;

    struct Trip
    {
```

```
    Zone zone;
    bool peak;
};

Trip trips[] = {
    {Zone::Inner, false},
    {Zone::Outer, true},
    {Zone::Middle, false}
};

for (const Trip& t : trips)
{
    long long f = fare(t.zone, t.peak);

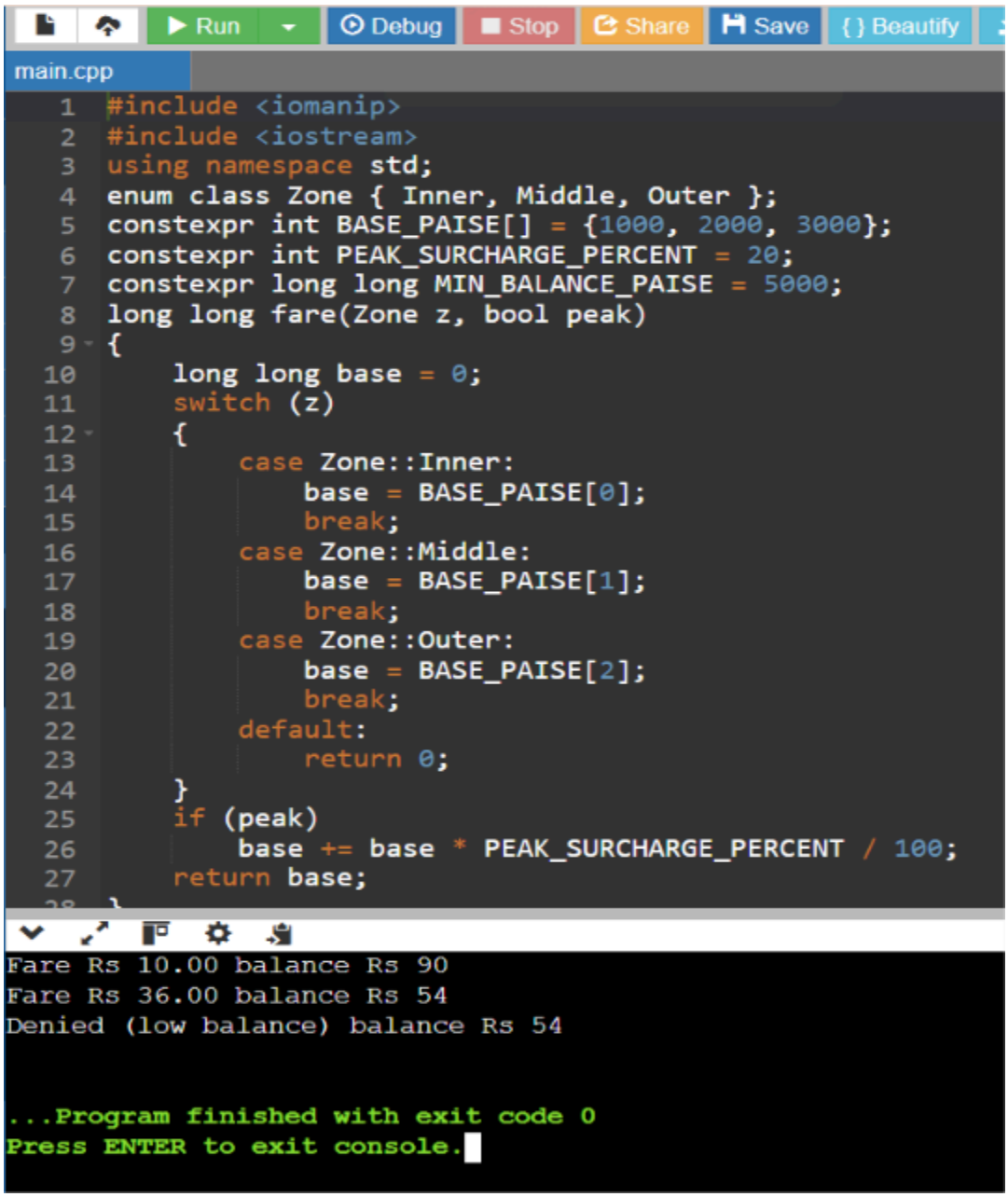
    if (canTravel(balance, f))
    {
        balance -= f;

        cout << "Fare Rs " << f / 100 << '.'
             << setw(2) << setfill('0') << f % 100
             << setfill(' ');
    }
    else
    {
        cout << "Denied (low balance)";
    }

    cout << " balance Rs " << balance / 100 << '\n';
}
```

```
    return 0;
}
```

Output



6. Peak Hour Boundary Testing

Hour	Peak Status	Reason
8 AM	Peak	Starting time included
10 AM	Non-peak	Ending time excluded
5 PM	Peak	Starting time included
8 PM	Non-peak	Ending time excluded

Correct peak expression:

```
None
(hour >= 8 && hour < 10) ||
(hour >= 17 && hour < 20)
```

7. Alternative Concept and Complexity Analysis

If an array is used instead of a switch statement:

```
None
constexpr long long FARE[] = {1000, 2000, 3000};
```

The base fare can be accessed using the zone index.

Feature	switch	Array
Lookup time	O(1)	O(1)
Extra storage	Usually none	O(z)
Fourth zone	Add another case	Add another array value

An array is easier to extend when more zones are added, while a switch provides clear decision logic.

8. Conclusion

Control structures are important for calculating fares, checking balances and processing journeys. A switch statement selects the zone fare, if-else checks travel eligibility, and loops process multiple journeys. Integer paise ensures accurate fare calculations.

Task 7 – From Procedural to Object-Oriented

1. Aim

To implement smart-card balance management using procedural and object-oriented programming and compare both methods.

2. Procedural Version

```
None

#include <iostream>
```

```
using namespace std;


const long long MIN_BALANCE = 5000; // Rs.50 in paise
bool canTravel(long long balance, long long fare)
{
   return balance - fare >= MIN_BALANCE;
}
void deduct(long long &balance, long long fare)
{
   if (canTravel(balance, fare))
      balance -= fare;
   else
      cout << "Journey Denied\n";
}
int main()
{
   long long balance = 10000; // Rs.100
   long long fare = 3000;    // Rs.30
```

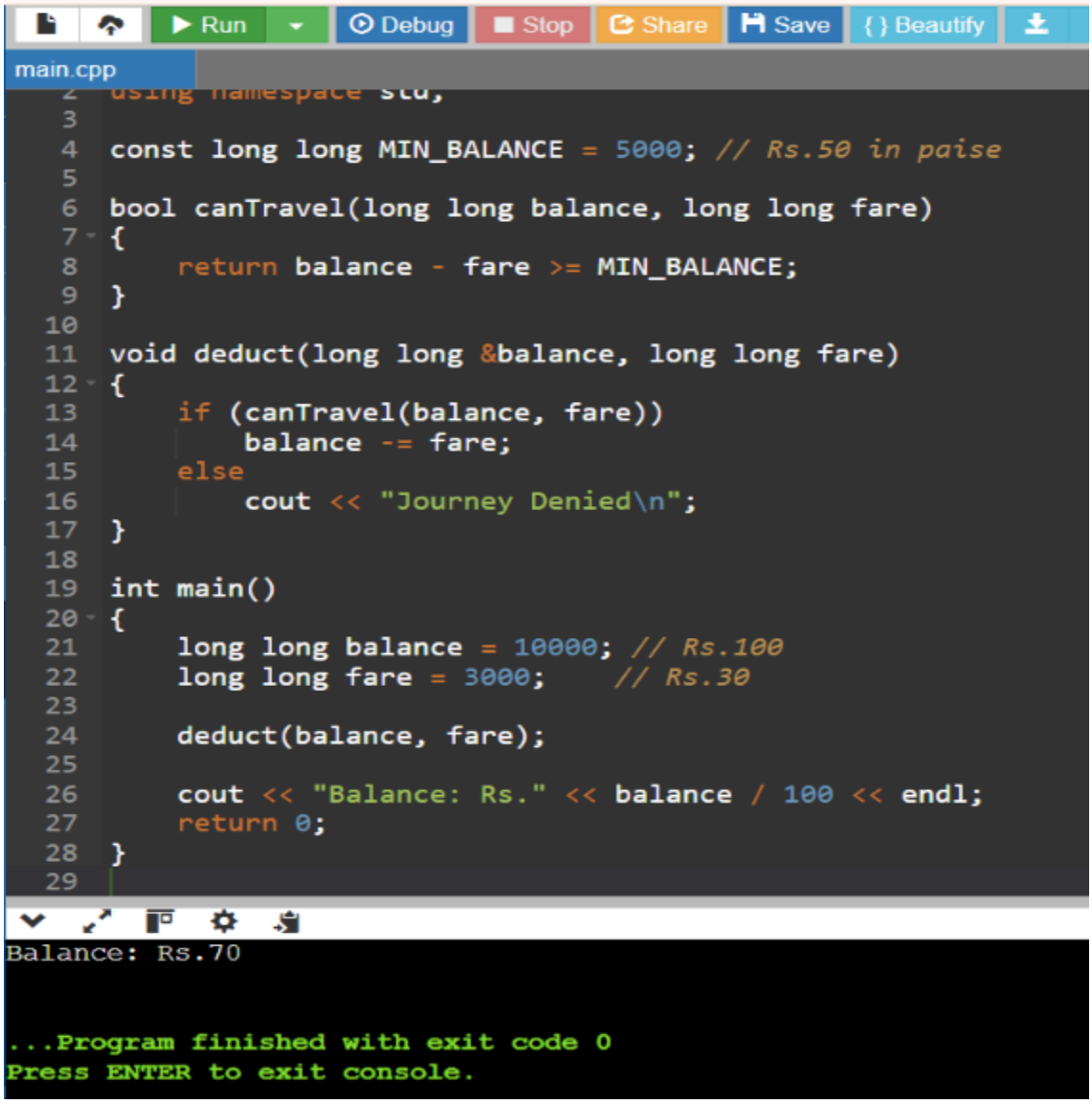
```
    deduct(balance, fare);

    cout << "Balance: Rs." << balance / 100 << endl;

    return 0;

}
```

Output:



3. Object-Oriented Version

None

```
#include <iostream>

using namespace std;

class SmartCard

{

private:

   long long balance;

   const long long MIN_BALANCE = 5000;

public:

   SmartCard(long long b)

   {

      balance = b;

   }

   bool canTravel(long long fare)

   {

      return balance - fare >= MIN_BALANCE;

   }
```

```
    void deduct(long long fare)
    {
        if (canTravel(fare))
        {
            balance -= fare;
            cout << "Journey Successful\n";
        }
        else
        {
            cout << "Journey Denied\n";
        }
    }
    long long getBalance()
    {
        return balance;
    }
};
int main()
{
```

```
    SmartCard card(10000); // Rs.100

    long long fare = 3000; // Rs.30

    card.deduct(fare);

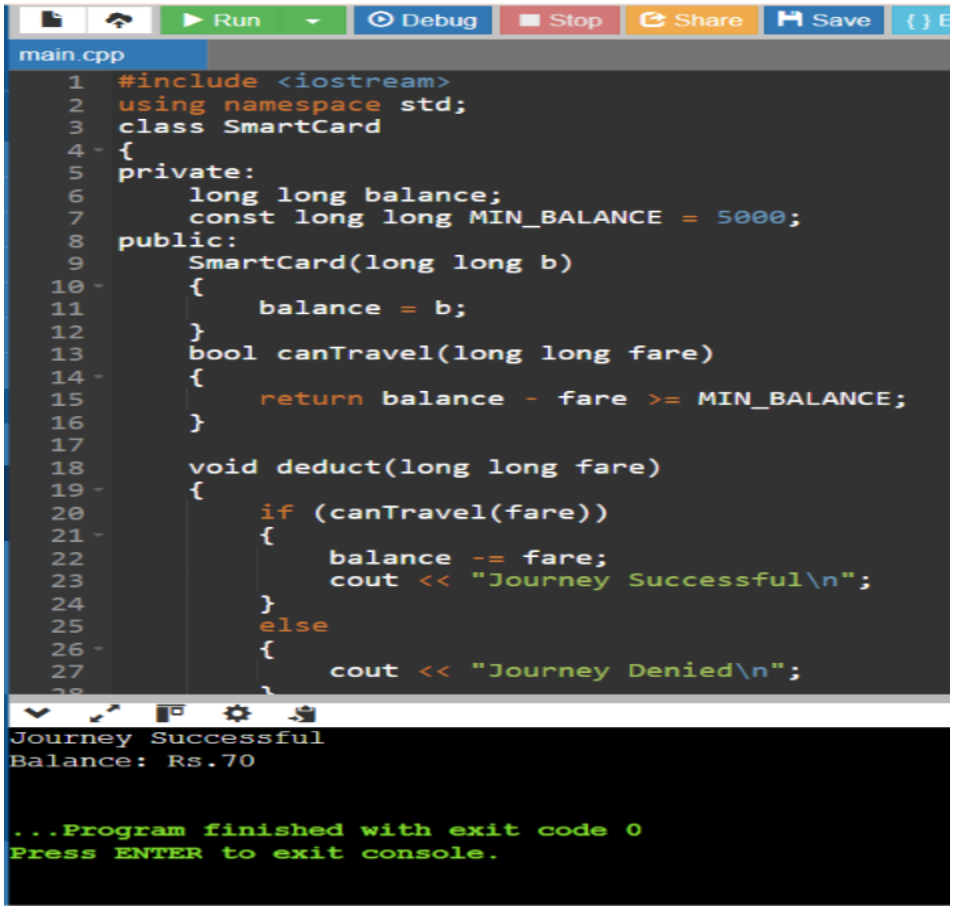
    cout << "Balance: Rs."

         << card.getBalance() / 100 << endl;

    return 0;

}
```

Output



5. Explanation

- Procedural programming: Uses separate variables and free functions like canTravel() and deduct().
- Object-oriented programming: Uses a SmartCard class containing balance and its related functions.
- Private balance: The balance cannot be directly modified outside the class.
- Prevention of negative balance: deduct() checks canTravel() before subtracting the fare.
- Minimum balance change: In OOP, changing MIN_BALANCE inside the class updates the rule centrally.

6. Alternative Concept – Public Balance

If balance is public:

```
None
class SmartCard

{

public:

    long long balance;

};
```

Any part of the program can modify the balance directly:

```
None
SmartCard card;

card.balance = -500;
```

This can violate the rule that balance must never become negative or fall below Rs.50 after a journey.

7. Complexity Analysis

Method	Time	Space
Procedural	O(1)	O(1)
OOP	O(1)	O(1)

Both methods have the same time and space complexity. OOP is safer because it uses encapsulation and protects the balance.

Conclusion: The object-oriented SmartCard design is better because it protects the balance and makes changes to fare rules easier.

Task 8 – Read and Print Data Safely

1. Aim

To read metro journey details safely using cin, handle invalid inputs, and display fare and balance in a proper format.

2. Algorithm

1. Start the program.
2. Read passenger name, zone, and fare.
3. Check whether input is valid using cin.fail().
4. If input is invalid, use cin.clear() and cin.ignore() to recover.
5. Calculate the remaining balance.
6. Display the journey report using <iomanip>.
7. Stop.

3. C++ Program

```
None
#include <iostream>

#include <iomanip>

#include <limits>

using namespace std;

int main()

{

   int zone;

   double fare, balance;

   cout << "Enter zone (1-3): ";

   cin >> zone;

   if (cin.fail())

   {

      cin.clear();

      cin.ignore(numeric_limits<streamsize>::max(), '\n');

      cout << "Invalid zone input!" << endl;

      return 0;

   }
```

```
cout << "Enter fare: ";
cin >> fare;
if (cin.fail())
{
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Invalid fare input!" << endl;
    return 0;
}
cout << "Enter balance: ";
cin >> balance;
if (cin.fail())
{
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Invalid balance input!" << endl;
    return 0;
}
```

```
    cout << "\n----- Journey Report -----\n";
    cout << "Zone       : " << zone << endl;
    cout << "Fare       : Rs. " << fixed << setprecision(2) << fare << endl;
    cout << "Balance    : Rs. " << fixed << setprecision(2) << balance << endl;
    return 0;
}
```

Output



Invalid Input Example

```
None
```

Enter zone (1-3): abc

Invalid zone input!

5. Explanation

- cin reads input from the user.
- cin.fail() checks whether input is invalid.
- cin.clear() clears the input error state.
- cin.ignore() discards incorrect input.
- setw() sets the output field width.
- setfill() fills empty spaces with a specified character.
- fixed and setprecision(2) display money with two decimal places.

6. Alternative Concept – getline() and stoi()

Instead of cin >>, we can use getline() to read a complete line and stoi() to convert it into an integer.

- getline() reads the entire input line.
- stoi() converts string into integer.
- try-catch handles invalid conversion.
- Processing a line of length L takes O(L) time.
- Buffer space required is O(L).

7. Complexity Analysis

Method	**Time**	**Space**

cin	O(1) for fixed-size input	O(1)
getline()	O(L)	O(L)

Conclusion: Safe input handling prevents incorrect data from affecting the fare system. Proper formatting makes the journey report clear and readable.

Task 9 – Implement and Test the Fare Logic

1. Aim

To implement and test the metro smart-card fare system using correct fare calculation, peak-hour rules, and minimum balance validation.

2. Algorithm

1. Start the program.
2. Read the zone and travel hour.
3. Calculate the base fare according to the zone.
4. Apply a 20% surcharge during peak hours (8–10 AM and 5–8 PM).
5. Check whether the remaining balance is at least Rs.50.
6. Deduct the fare if the journey is allowed.
7. Display fare and balance.
8. Repeat for five journeys.
9. Stop.

3. C++ Program

```
None

#include <iostream>
```

```
#include <iomanip>

using namespace std;

const long long MIN_BALANCE = 5000;

long long calculateFare(int zone, int hour)

{

   long long fare;

   switch (zone)

   {

      case 1: fare = 1000; break;

      case 2: fare = 2000; break;

      case 3: fare = 3000; break;

      default: return -1;

   }

   bool peak = (hour >= 8 && hour < 10) ||

            (hour >= 17 && hour < 20);

   if (peak)

      fare += fare * 20 / 100;

   return fare;
```

```
}

bool canTravel(long long balance, long long fare)

{

   return balance - fare >= MIN_BALANCE;

}

int main()

{

   long long balance = 10000;

   int zone, hour;

   for (int i = 1; i <= 5; i++)

   {

      cout << "\nJourney " << i << endl;

      cout << "Enter zone (1-3): ";

      cin >> zone;

      if (cin.fail())

      {

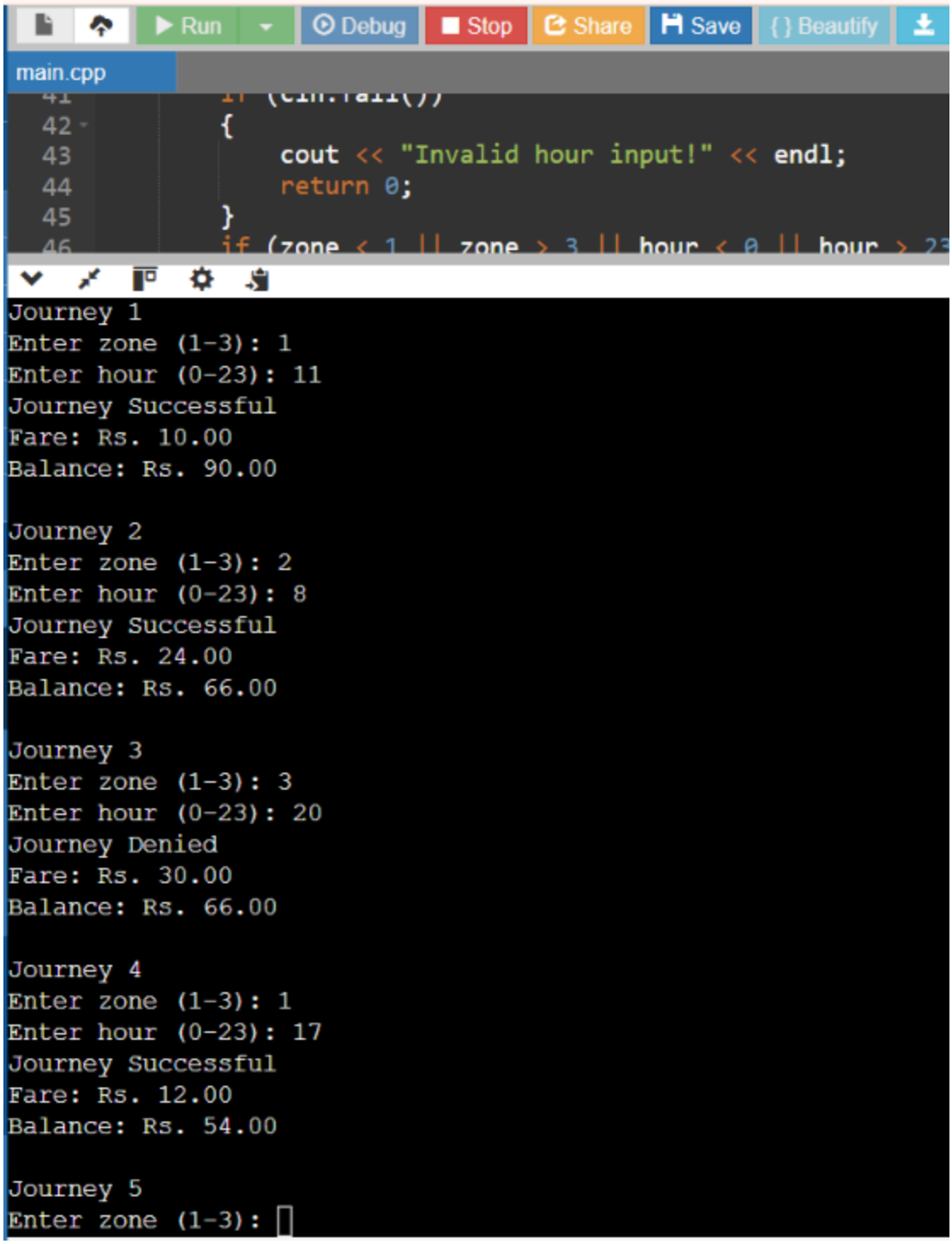
         cout << "Invalid zone input!" << endl;

         return 0;
```

```
}
cout << "Enter hour (0-23): ";
cin >> hour;
if (cin.fail())
{
    cout << "Invalid hour input!" << endl;
    return 0;
}
if (zone < 1 || zone > 3 || hour < 0 || hour > 23)
{
    cout << "Invalid input!" << endl;
    continue;
}
long long fare = calculateFare(zone, hour);
if (canTravel(balance, fare))
{
    balance -= fare;
    cout << "Journey Successful" << endl;
```

```
        }
        else
        {
            cout << "Journey Denied" << endl;
        }
        cout << "Fare: Rs. " << fixed << setprecision(2)
            << fare / 100.0 << endl;
        cout << "Balance: Rs. " << fixed << setprecision(2)
            << balance / 100.0 << endl;
    }
    return 0;
}
```

Output


Run
Debug
Stop
Share
Save
{ } Beautify
main.cpp
{
cout << "Invalid hour input!" << endl;
return 0;
}
Journey 1
Enter zone (1-3): 1
Enter hour (0-23): 11
Journey Successful
Fare: Rs. 10.00
Balance: Rs. 90.00
Journey 2
Enter zone (1-3): 2
Enter hour (0-23): 8
Journey Successful
Fare: Rs. 24.00
Balance: Rs. 66.00
Journey 3
Enter zone (1-3): 3
Enter hour (0-23): 20
Journey Denied
Fare: Rs. 30.00
Balance: Rs. 66.00
Journey 4
Enter zone (1-3): 1
Enter hour (0-23): 17
Journey Successful
Fare: Rs. 12.00
Balance: Rs. 54.00
Journey 5
Enter zone (1-3):

5. Test Cases

Test	Input	Expected Result
Inner off-peak	Zone 1, 11 AM	Rs.10
Middle peak	Zone 2, 8 AM	Rs.24
Outer off-peak	Zone 3, 12 PM	Rs.30
Exactly 10 AM	Zone 1, 10 AM	Rs.10
Exactly 17 PM	Zone 1, 17 PM	Rs.12
Exactly 20 PM	Zone 1, 20 PM	Rs.10
Unknown zone	Zone 4	Invalid input
Letters for hour	abc	Invalid input
Balance leaves Rs.50	Balance Rs.60, fare Rs.10	Allowed
Negative recharge	Recharge -100	Reject

6. Alternative Concept – Test Case Table

Instead of writing separate test statements, store test cases in an array and execute them using a loop.

- Each test contains zone, hour, expected fare, and expected result.
- The loop checks every test automatically.
- A new boundary test can be added easily.
- For t test cases, time complexity is O(t).
- Space complexity for the test table is O(t).

7. Complexity Analysis

Operation	**Time Complexity**
Fare calculation	O(1)
Balance check	O(1)
Five journeys	O(5) = O(1)
t test cases	O(t)

Conclusion: Integer paise calculation avoids rounding errors, and boundary testing ensures the peak-hour and minimum-balance rules work correctly.

Task 10 – Compare Expectations with Execution

1. Aim

To compare expected and actual fare totals using double, integer paise, and int, and identify precision and overflow problems.

2. C++ Program

```
None
#include <iostream>

#include <iomanip>

using namespace std;

int main()

{

   double fareRupees = 10.01;

   long long farePaise = 1001;

   int journeys[] = {20000, 500000, 3000000};

   cout << fixed << setprecision(2);

   for (int n : journeys)

   {

      double doubleTotal = fareRupees * n;

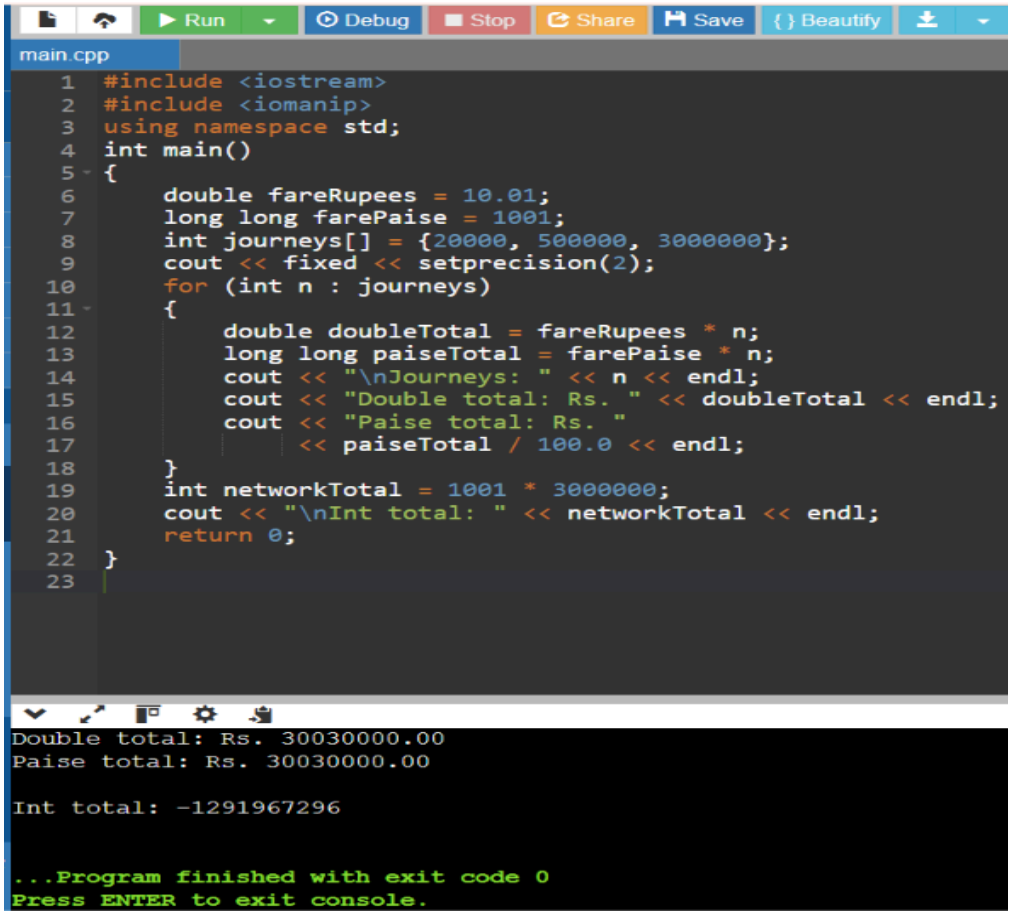
      long long paiseTotal = farePaise * n;

      cout << "\nJourneys: " << n << endl;

      cout << "Double total: Rs. " << doubleTotal << endl;
```

```
        cout << "Paise total: Rs. "
             << paiseTotal / 100.0 << endl;
    }
    int networkTotal = 1001 * 3000000;
    cout << "\nInt total: " << networkTotal << endl;
    return 0;
}
```

Output:



3. Expected Results

Journeys	Double Total (Rs.)	Integer Paise Total (Rs.)
20,000	200,200.00	200,200.00
500,000	5,005,000.00	5,005,000.00
3,000,000	30,030,000.00	30,030,000.00

4. Explanation

- double stores money in floating-point format, so small rounding errors may occur.
- Integer paise stores money exactly without decimal rounding.
- int has a maximum value of approximately 2,147,483,647.
- Network total in paise is 3,003,000,000, which exceeds the int range.
- long long can safely store this total.

5. Compiler Warnings

Compile using:

```
None
g++ -std=c++17 -Wall -Wextra task10.cpp -o task10
```

Possible warnings:

- -Wall: Common programming warnings.
- -Wextra: Additional warnings.
- Signed/unsigned comparisons or unused variables may produce warnings.

- Runtime integer overflow is not necessarily detected by these warning flags.

6. Alternative Concept – Data Types

Data Type	Time per Addition	Space	Limitation
int	O(1)	4 bytes	Overflow
long long	O(1)	8 bytes	Very large limit
double	O(1)	8 bytes	Precision errors

7. Conclusion

Integer paise with long long is the safest choice for metro fare calculations because it avoids rounding errors and supports large network totals without overflow.

Task 11 – Final Metro Smart-Card System Design

1. Aim

To recommend a final metro smart-card system design using suitable data types, constants, classes, and fare calculation methods.

2. Recommended Design

Component	Recommended choice	Reason

Zone	enum class Zone	Type-safe zone identification
Fare	long long in paise	Avoids decimal rounding
Fare constants	constexpr	Compile-time fixed values
Fare table	Array	Easy to update and extend
SmartCard	Class	Protects balance using encapsulation
Balance	Private variable	Prevents direct unauthorized modification
Fare calculation	Separate function	Easy to test and maintain

3. Final C++ Design

```
None
#include <iostream>

#include <iomanip>

using namespace std;

enum class Zone { Inner, Middle, Outer };

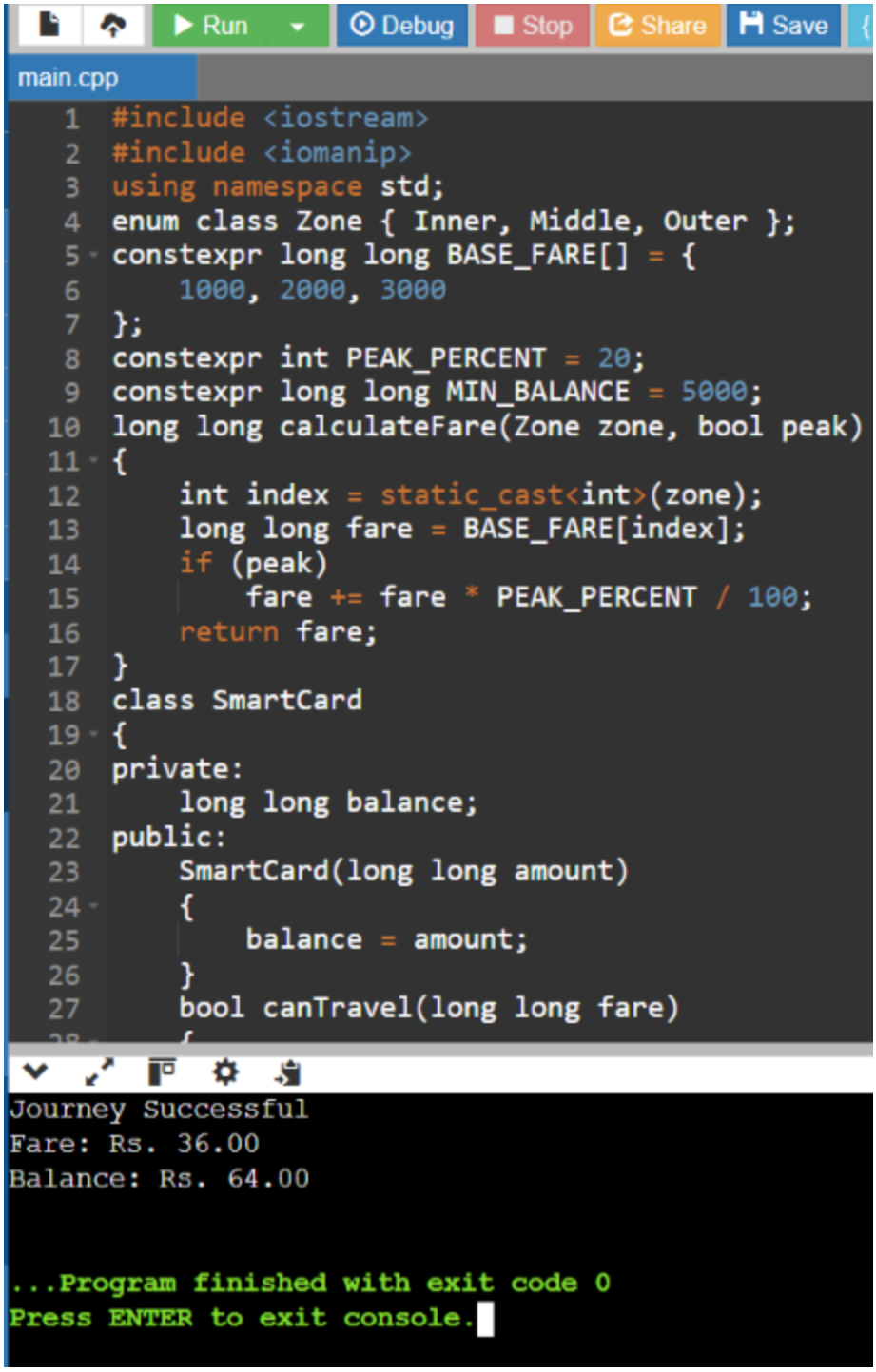
constexpr long long BASE_FARE[] = {
```

```
    1000, 2000, 3000
};
constexpr int PEAK_PERCENT = 20;
constexpr long long MIN_BALANCE = 5000;
long long calculateFare(Zone zone, bool peak)
{
    int index = static_cast<int>(zone);
    long long fare = BASE_FARE[index];
    if (peak)
        fare += fare * PEAK_PERCENT / 100;
    return fare;
}
class SmartCard
{
private:
    long long balance;
public:
    SmartCard(long long amount)
    {
```

```
    balance = amount;
}
bool canTravel(long long fare)
{
    return balance - fare >= MIN_BALANCE;
}
void deductFare(long long fare)
{
    if (canTravel(fare))
    {
        balance -= fare;
        cout << "Journey Successful\n";
    }
    else
    {
        cout << "Journey Denied\n";
    }
}
long long getBalance()
```

```
    {
        return balance;
    }
};
int main()
{
    SmartCard card(10000);
    long long fare = calculateFare(Zone::Outer, true);
    card.deductFare(fare);
    cout << fixed << setprecision(2);
    cout << "Fare: Rs. " << fare / 100.0 << endl;
    cout << "Balance: Rs. "
         << card.getBalance() / 100.0 << endl;
    return 0;
}
```

Output



4. Fare Revision Process

Two methods can be used:

Method 1: Compile-time constants

- Fare values are stored using constexpr.
- The program must be recompiled whenever fares change.
- Suitable for small systems with fixed fare rules.

Method 2: Configuration file

- Fare values are stored in an external file.
- The program reads fares during startup.
- Fare changes do not require recompilation.
- Suitable for large metro networks with frequent fare changes.

Comparison

Feature	**constexpr**	**Configuration file**
Fare modification	Recompile	Edit file
Startup time	Faster	File loading required
Runtime flexibility	Low	High
Memory	Small	Table storage
Maintenance	Less flexible	Easier

5. Experimental Evidence

Experiment	Observation
Integer paise	Exact fare calculations
Floating-point fare	Possible rounding errors
SmartCard class	Protects balance
Minimum balance check	Prevents invalid journeys
Fare array	Simplifies fare revision
Peak surcharge	Correctly calculates 20%

6. Limitations

- Compile-time constants require recompilation for fare changes.
- Configuration files require validation and error handling.
- The system needs additional logic for concession eligibility.
- Invalid zone values must be checked.
- Large networks require database integration.

7. Conclusion

The recommended metro smart-card design uses enum class, constexpr, integer paise, arrays, and an encapsulated SmartCard class. It provides accurate fare calculations, secure balance

management, and easy maintenance. A configuration-based fare table is more suitable for a large metro network because fare changes can be made without recompiling the program.

Task 12 – Revised Metro Smart-Card System

A. Impact Analysis

The metro system introduces three new requirements:

1. Airport Zone: A new zone with a base fare of Rs.60.
2. Weekend Discount: A 10% discount applies on Saturday and Sunday. It cannot be combined with the peak surcharge.
3. Monthly Pass: Passengers can purchase unlimited journeys within two selected zones.

Revised Fare Rules

Zone	**Base Fare**
Inner	Rs.10
Middle	Rs.20
Outer	Rs.30
Airport	Rs.60

- Peak surcharge: 20%
- Weekend discount: 10%

- Weekend discount takes priority over peak surcharge.
- Minimum remaining card balance: Rs.50.
- Monthly pass allows unlimited travel in two selected zones.

B. Evaluation of Existing Design

Existing Design	**Impact**	**Required Change**
Three-zone switch	Airport needs another case	Add Airport zone
Fare array	Easy to extend	Add fourth fare
SmartCard class	Suitable	Add pass support
Peak calculation	Weekend conflict	Add weekend condition
Balance check	Suitable	Maintain Rs.50 minimum

Best approach: Use a fare array and a SmartCard class because they are easier to extend and maintain.

C. Revised C++ Implementation

```
None
#include <iostream>

#include <iomanip>
```

```
using namespace std;

enum class Zone { Inner, Middle, Outer, Airport };

constexpr long long BASE_FARE[] = {

    1000, 2000, 3000, 6000

};

constexpr int PEAK_PERCENT = 20;

constexpr int WEEKEND_DISCOUNT = 10;

constexpr long long MIN_BALANCE = 5000;

long long calculateFare(Zone zone, bool peak, bool weekend)

{

    int index = static_cast<int>(zone);

    long long fare = BASE_FARE[index];

    if (weekend)

    {

        fare -= fare * WEEKEND_DISCOUNT / 100;

    }

    else if (peak)

    {

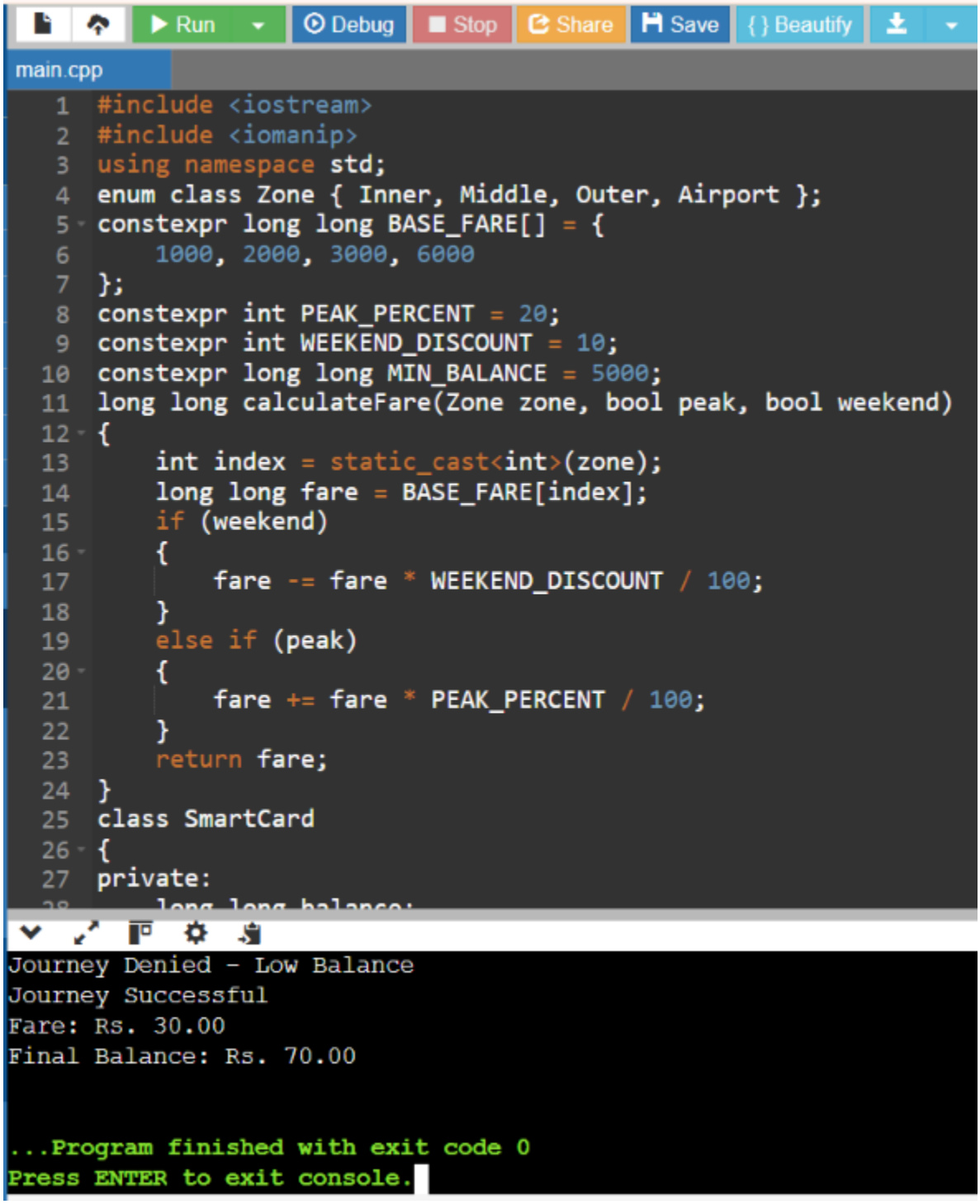
        fare += fare * PEAK_PERCENT / 100;
```

```
    }
    return fare;
}
class SmartCard
{
private:
    long long balance;
    bool monthlyPass[4];
public:
    SmartCard(long long amount)
    {
        balance = amount;
        for (int i = 0; i < 4; i++)
            monthlyPass[i] = false;
    }
    void activatePass(Zone z1, Zone z2)
    {
        monthlyPass[static_cast<int>(z1)] = true;
        monthlyPass[static_cast<int>(z2)] = true;
```

```
}
void travel(Zone zone, bool peak, bool weekend)
{
    int index = static_cast<int>(zone);
    if (monthlyPass[index])
    {
        cout << "Monthly Pass Used - Journey Successful\n";
        return;
    }
    long long fare = calculateFare(zone, peak, weekend);
    if (balance - fare >= MIN_BALANCE)
    {
        balance -= fare;
        cout << "Journey Successful\n";
        cout << "Fare: Rs. " << fare / 100.0 << endl;
    }
    else
    {
        cout << "Journey Denied - Low Balance\n";
```

```
        }
    }
    long long getBalance()
    {
        return balance;
    }
};
int main()
{
    SmartCard card(10000);
    card.activatePass(Zone::Inner, Zone::Middle);
    cout << fixed << setprecision(2);
    card.travel(Zone::Inner, false, false);
    card.travel(Zone::Airport, true, true);
    card.travel(Zone::Outer, false, false);
    cout << "Final Balance: Rs. "
         << card.getBalance() / 100.0 << endl;
    return 0;
}
```

Output



Correction to the example: The third journey is denied because Rs.16 is below the required Rs.50 minimum. Therefore, the accurate final output is:

D. Evidence-Based Design Justification

Feature	**Reason**
enum class Zone	Type-safe zone selection
Fare array	Supports four zones easily
Integer paise	Avoids floating-point fare errors
SmartCard class	Protects balance
Monthly pass array	Identifies eligible zones

Complexity Analysis

- Fare calculation: O(1)
- Monthly pass checking: O(1)
- Journey processing: O(1)
- Monthly pass storage: O(4), constant space

Conclusion

The revised system supports Airport travel, weekend discounts, peak surcharges, and monthly passes for two selected zones. The array-based fare table and SmartCard class make the system simple, accurate, and expandable.